PYTHON
to MACHINE LEARNING

Complete study notes - from `print()` *to gradient descent*

18 topics, every example executed and verified, with extra explanations, tables, traps and practice tasks added to the original handwritten-style notes.

Your roadmap

1–8 Core Python	**9–13** Data structures	**14** Files	**15–16** OOP	**17** Regression	**18** Calculus & gradient descent

How to read these notes - the sticky-note legend

Key idea:	the idea to remember if you remember nothing else
Careful:	a trap that costs beginners hours
Why it works:	the reason behind the rule - understand, don't memorise
Trick:	a shortcut or a way to remember
Note-fix	where the original notes were simplified and this adds precision
Try it yourself:	exercises - type them, change the numbers, break them

Study tip from the original notes: *type every example yourself and change the numbers - that is the fastest way to learn. Yellow boxes are code (comments in green); the green boxes marked* ***output:*** *are what the code really printed when it was run.*

Contents

1. Python Basics 3

2. Variables 6

3. Data Types 8

4. Strings 11

5. Operators 15

6. Conditional Statements 18

7. Loops 21

8. Functions 25

9. List 29

10. Array (and NumPy) 32

11. Tuple 35

12. Dictionary 37

13. Set 39

14. File Handling 41

15. OOP: Class and Object 44

16. The 4 Pillars of OOP 47

17. Regression 52

18. Derivative, Chain Rule & Gradient Descent 57

19. Quick Revision Sheet 61

20. Practice Problems 63

Last pages: Quick Revision Sheet - Common Mistakes - Practice Problems with answers - 4-week study plan.

1 Python Basics

Meet the language: what it is, how it runs, and how to talk to it

Python is a **high-level, general-purpose programming language** created by Guido van Rossum (first released in 1991). Its syntax is deliberately close to plain English, which is why it is the most popular first language and, at the same time, the main language of AI / machine learning.

Why Python? (the real reasons)

- **Readable** - fewer symbols than C/Java; no { } or ; clutter. Code is read far more often than it is written.
- **Huge ecosystem** - NumPy, pandas, scikit-learn, PyTorch, TensorFlow (AI/ML); Django, Flask (web); Selenium (automation).
- **Productive** - a task that takes 40 lines in Java often takes 5-10 in Python.
- **Cross-platform & free** - the same code runs on Windows, Linux and macOS.

Four ways to run Python

Way	How	Best for
Script file	Save as name.py, run python name.py in a terminal	Real programs, anything you want to keep
Interactive shell (REPL)	Type python, then type code and press Enter	Quick experiments, using Python as a calculator
Notebook (Jupyter / Colab)	Run code in cells, see results and charts under each cell	Data science and ML work
IDE / editor	VS Code, PyCharm - write, run and debug in one place	Bigger projects

"Interpreted" - what really happens

The original notes say Python runs your code line by line. That is the right mental model for a beginner, and here is the slightly more exact picture:

name.py (your source)	→ compile →	bytecode (.pyc in __pycache__)	→ run →	Python Virtual Machine	→	output

- You never run a separate compile command - Python does it silently each time.
- Because execution is step by step, a mistake on line 50 is only noticed **when Python reaches line 50** (except syntax errors, which are caught before anything runs).

```
print("before")
print(1 / 0)        # runtime error: happens only when this line is reached
print("after")
```

output:

```
before
Traceback (most recent call last):
  File "demo.py", line 2, in <module>
    print(1 / 0)
ZeroDivisionError: division by zero
```

Note-fix (more precise than the original): "Interpreted" is a property of the implementation (CPython), not of the language itself. CPython compiles to bytecode first; that is why a Python program that is syntactically wrong never prints even its first line, while a runtime error prints everything before it.

print() in depth

print() sends text to the screen. It can take many values, and two useful options: sep (what goes between values, default a space) and end (what goes after, default a new line).

```
print("Hello, World!")          # text must be inside quotes
print("a", "b", "c")            # several values -> separated by a space
print("a", "b", "c", sep="-")   # change the separator
print("Loading", end="...")     # end="" or end="..." stops the new line
print("done")
print(3 + 4)                    # Python calculates first -> 7
print("3 + 4")                  # inside quotes it is just text
print("Line1\nLine2")           # \n = new line character
print()                         # empty line
```

output:

```
Hello, World!
a b c
a-b-c
Loading...done
7
3 + 4
Line1
Line2
```

Comments & docstrings

- A # starts a ***comment***: Python ignores everything after it on that line. Use comments to explain *why*, not to repeat *what*.
- A ***docstring*** is a triple-quoted string placed as the first line inside a function/class/file; tools like help() display it.
- Shortcut: select lines and press Ctrl + / in most editors to comment them out while debugging.

```
# this whole line is a comment
price = 250                                  # inline comment: price in rupees

def area(r):
    """Return the area of a circle of radius r."""
    return 3.14159 * r * r

print(area.__doc__)
```

output:

```
Return the area of a circle of radius r.
```

Syntax rules you must remember

- ***Indentation is syntax.*** A block (after :) is marked by indenting - 4 spaces is the standard. Mixing tabs and spaces breaks the code.
- ***Case sensitive:*** print works, Print gives a NameError.
- ***One statement per line*** (use ; or line breaks inside ( ) only if you really need to).
- Long lines can continue inside brackets, or with a trailing backslash \.

Error name	Typical cause	Example
SyntaxError	Missing quote / bracket / colon	`print("hi)`
IndentationError	Wrong or missing indentation	body of `if` not indented
NameError	Using a name that does not exist (typo)	`prnt(5)`, using a variable before assigning
TypeError	Operation on the wrong type	`"5" + 5`
ZeroDivisionError	Dividing by zero	`10 / 0`

Trick: Read a traceback from the **bottom up**: the last line tells you what went wrong, the lines above tell you where.

Try it yourself:

- Print your name and age on two separate lines, then on one line separated by `|`.
- Make Python print `Hi!` three times on one line using `end`.
- Deliberately create a NameError, a SyntaxError and a TypeError, and read each message.

2 Variables

Names that point to values - the memory of your program

A **variable** is a name that refers to a value stored in memory. The notes describe it as a named box; that picture works at first, but a more accurate picture is a **name tag attached to a value**. Several tags can be attached to the same value, and moving a tag never changes the value itself.

```
a = 10
b = a                   # b is another tag on the SAME value 10
print(id(a) == id(b))   # True: same object in memory
a = 20                  # tag 'a' moves to a new value; 'b' is unaffected
print(a, b)
```

output:
```
True
20 10
```

Creating variables - Python is dynamically typed

You never declare a type. The assignment = creates the variable, and the **type belongs to the value, not to the name**, so one name may point to different types over time.

```
name = "Ali"        # str
age = 20            # int
height = 5.9        # float
is_student = True   # bool
x = 5
print(type(x))
x = "five"          # allowed - same name, new value, new type
print(type(x))
```

output:
```
<class 'int'>
<class 'str'>
```

Multiple assignment & swapping

```
a, b = 1, 2         # two variables in one line
x = y = 0           # same value to both
print(a, b, x, y)
a, b = b, a         # classic swap - no temporary variable needed in Python
print(a, b)
```

output:
```
1 2 0 0
2 1
```

Naming rules (must) and conventions (should)

Rule	Valid	Invalid
Start with a letter or _, never a digit	score1, _temp	1score
Only letters, digits and _ - no spaces or symbols	total_marks	total marks, my-age, price$
Case matters	Age and age are two different variables	-
Cannot be a keyword	class_name, for_sale	class, for, if

PEP 8 convention	Used for	Example
snake_case	variables, functions	student_age, get_total()
PascalCase	classes	BankAccount
UPPER_CASE	constants (by convention only - Python does not enforce)	PI = 3.14159, MAX_USERS = 100

Keywords - words Python reserves

```
import keyword
print(len(keyword.kwlist))      # how many keywords this Python has
print(keyword.kwlist[:10])      # the first ten
```

output:

```
35
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class']
```

Deleting a variable

```
score = 90
del score                       # removes the name
try:
    print(score)
except NameError as e:
    print("NameError:", e)
```

output:

```
NameError: name 'score' is not defined
```

Careful:

- Never use built-in names like list, str, sum, max, input as variable names - list = [1,2] silently breaks list() for the rest of the program.
- A variable must be **assigned before use**; using it earlier gives a NameError.

Try it yourself:

- Swap two variables holding your height and weight without a third variable.
- Make b = a where a is a list, change b, and observe a (preview of aliasing in Topic 9).

3 Data Types

What kind of value is it? Numbers, text, truth values, and nothing

Type	Meaning	Example	Notes
int	whole number	10, -3, 0	no size limit in Python
float	decimal number	3.14, -0.5, 2e3	stored in binary - tiny rounding errors
str	text	"hello", 'hi'	immutable sequence of characters
bool	True / False	True, False	capital T/F; behaves like 1 and 0
NoneType	nothing / empty	None	the only value of its type
complex	complex number	3+4j	rare; used in engineering

Containers (list, tuple, dict, set) are also data types - they get their own topics later. Use type(x) to see a type and isinstance(x, int) to test it.

```
x = 10;  y = 3.14;  s = "hello";  ok = True;  n = None
for v in (x, y, s, ok, n):
    print(repr(v), "->", type(v).__name__)
print(isinstance(x, int), isinstance(ok, int))          # bool counts as int!
```

output:

```
10 -> int
3.14 -> float
'hello' -> str
True -> bool
None -> NoneType
True True
```

Things that surprise beginners

```
print(2 ** 100)              # int never overflows
print(0.1 + 0.2)             # 0.30000000000000004  (binary fractions)
print(round(0.1 + 0.2, 2))   # 0.3 - round when you display
print(True + True)           # 2 - bool is a subclass of int
print(10 / 2, type(10 / 2))  # / always gives float
```

output:

```
1267650600228229401496703205376
0.30000000000000004
0.3
2
5.0 <class 'float'>
```

Why it works: *Computers store floats in binary, and just as 1/3 cannot be written exactly in decimal, 0.1 cannot be written exactly in binary. That is* ***not a Python bug*** *- every language does this. Compare floats with* `math.isclose(a, b)` *instead of* `==`.

None – the value that means "nothing here"

- A function with no return gives back None.
- Test with is None (not == None).
- None is not the same as 0, "" or False – it means "no value at all".

Type conversion (casting)

```
print(int("5"), int(3.9), int(-3.9))          # int() cuts the decimals, it does not round
print(float("2.5"), float(7))
print(str(100) + "%")                         # number -> text
print(round(2.5), round(3.5), round(3.14159, 2))
print(bool(0), bool(""), bool("False"), bool([0]))
try:
    int("3.5")                                # text with a decimal point is not an int
except ValueError as e:
    print("ValueError:", e)
```

output:

```
5 3 -3
2.5 7.0
100%
2 4 3.14
False False True True
ValueError: invalid literal for int() with base 10: '3.5'
```

Key idea: round() *uses* ***banker's rounding*** *(ties go to the even number):* round(2.5) *is 2 and* round(3.5) *is 4. It is intentional and reduces bias in statistics.*

Truthy and falsy values

In if *and* while*, Python converts the value to a bool. These are* ***falsy****; everything else is* ***truthy****:*

Falsy	False	None	0, 0.0	"" *(empty string)*	[] () {} set() *(empty containers)*

input() always returns text

```
age = input("Age: ")          # the user types 20
print(type(age))              # <class 'str'>
print(age + 1)                # TypeError: can only concatenate str to str
age = int(age)                # convert first...
print(age + 1)                # ...then do maths -> 21
```

output:

```
Age: 20
<class 'str'>
TypeError: can only concatenate str (not "int") to str
21
```

```
# safe pattern: keep asking until the user types a valid number
while True:
    text = input("Enter marks: ")
    try:
        marks = float(text)
```

```
        break
    except ValueError:
        print("Please type a number!")
```

Mixing types in expressions

```
print(5 + 2.0)          # int + float -> float
print("5" * 3)          # str * int -> repeats the text
try:
    print("5" + 5)
except TypeError as e:
    print("TypeError:", e)
print(int("5") + 5)
```

output:

```
7.0
555
TypeError: can only concatenate str (not "int") to str
10
```

Try it yourself:

- *Predict, then check:* `bool("0")`, `bool(0.0)`, `int(True)`, `10 // 4`, `type(10 / 5)`.
- *Write a program that asks for two numbers and prints their sum (remember to convert!).*

4 Strings

Text in Python: indexing, slicing, methods and f-strings

A **string** (str) is an ordered sequence of characters written inside single, double or triple quotes. Because it is a sequence, everything you learn here (indexing, slicing, len, in, looping) also works for lists and tuples.

Index map of "Python"

character	P	y	t	h	o	n
index (from left)	0	1	2	3	4	5
index (from right)	-6	-5	-4	-3	-2	-1

Indexing and slicing: s[start : stop : step]

start is included, stop is **excluded** ("up to but not including"), step is the jump. Any part can be omitted.

```
s = "Python"
print(s[0], s[-1])      # P n
print(s[0:3], s[:3])    # Pyt Pyt
print(s[3:], s[-3:])    # hon hon
print(s[::2])           # Pto  (every 2nd character)
print(s[::-1])          # nohtyP  (reversed - the famous trick)
print(s[2:100])         # thon  (slices never raise IndexError)
print(len(s))           # 6
```

output:

```
P n
Pyt Pyt
hon hon
Pto
nohtyP
thon
6
```

Careful: s[10] raises **IndexError**, but the slice s[10:] quietly returns ''. Indexing is strict; slicing is forgiving.

Strings are immutable

You cannot change a character in place. To "modify" a string you build a **new** one.

```
s = "Python"
try:
    s[0] = "J"
except TypeError as e:
    print("TypeError:", e)
s = "J" + s[1:]                 # build a new string instead
print(s)                        # Jython
print("Python".replace("P", "J"))   # replace also returns a NEW string
```

output:

```
TypeError: 'str' object does not support item assignment
Jython
Jython
```

The essential methods

Method	What it does	Example → result
upper() / lower()	change case	"Py".upper() → 'PY'
title() / capitalize()	Title Case / first letter only	"hi all".title() → 'Hi All'
strip(), lstrip(), rstrip()	remove spaces/newlines at the ends	" hi ".strip() → 'hi'
replace(old, new)	replace every occurrence	"aab".replace("a","x") → 'xxb'
split(sep)	string → list	"a,b,c".split(",") → ['a','b','c']
"sep".join(list)	list → string	"-".join(["a","b"]) → 'a-b'
find(x) / index(x)	position of x (find gives -1 if absent, index raises error)	"hello".find("l") → 2
count(x)	how many times x occurs	"banana".count("a") → 3
startswith() / endswith()	check beginning / end	"file.py".endswith(".py") → True
isdigit(), isalpha(), isalnum()	check what the characters are	"123".isdigit() → True

```
text = "  Hello, Python World  "
clean = text.strip()
print(clean.upper(), "|", clean.lower())
words = clean.split(" ")
print(words, len(words))
print("_".join(words))
print(clean.count("o"), clean.find("Python"), clean.startswith("Hello"))
print("a,b,c".split(","))
```

output:

```
HELLO, PYTHON WORLD | hello, python world
['Hello,', 'Python', 'World'] 3
Hello,_Python_World
3 7 True
['a', 'b', 'c']
```

Joining, repeating, escaping, multi-line

```
print("Py" + "thon", "ha" * 3)              # + joins, * repeats
print("She said \"hi\"")                    # \" puts a quote inside a string
print("Name:\tAli\nAge:\t20")               # \t = tab, \n = new line
print(r"C:\new\table")                      # raw string: backslashes stay as typed
poem = """Roses are red,
violets are blue"""                         # triple quotes allow many lines
print(poem)
print("py" in "python", "z" not in "python")  # membership
```

output:

```
Python hahaha
She said "hi"
Name:	Ali
Age:	20
C:\new\table
Roses are red,
violets are blue
True True
```

Escape	Meaning	Escape	Meaning
\n	new line	\\	one backslash
\t	tab	\' \"	quote inside a string

f-strings – the modern way to build text

Put an f before the quote and write variables or whole expressions inside { }. After a colon you can add a **format spec**.

```
name, age, pi = "Ali", 20, 3.14159265
price, big = 7, 1234567.891
print(f"My name is {name} and I am {age}")
print(f"Next year I will be {age + 1}")          # any expression works
print(f"pi = {pi:.2f}")                          # 2 decimal places
print(f"{price:05d}|{big:,.1f}")                 # zero-padded | thousands separator
print(f"[{name:>8}] [{name:<8}] [{name:^8}]")    # right / left / centre in width 8
print(f"{0.256:.1%}")                            # percent
print(f"{age=}")                                 # debug form: shows name and value
```

output:

```
My name is Ali and I am 20
Next year I will be 21
pi = 3.14
00007|1,234,567.9
[    Ali] [Ali     ] [  Ali   ]
25.6%
age=20
```

Key idea: Older alternatives you may meet in other code: `"{} is {}".format(name, age)` and `"%s is %d" % (name, age)`. Prefer f-strings.

Typical string programs

```
word = "Level"
print(word.lower() == word.lower()[::-1])        # palindrome check -> True
sentence = "Python is fun and python is easy"
print(sentence.lower().count("python"))          # case-insensitive count -> 2
vowels = sum(1 for ch in "education" if ch in "aeiou")
print(vowels)                                    # 5
for i, ch in enumerate("abc"):                   # looping over characters
    print(i, ch)
```

output:

```
True
2
5
0 a
1 b
2 c
```

Try it yourself:

- *Reverse each word of a sentence but keep the word order.*
- *Count how many times each letter appears in "mississippi" (hint: you will use a dict in Topic 12).*
- *Format 1234.5 as Rs. 1,234.50 using an f-string.*

5 Operators

Arithmetic, comparison, logic, membership - and who goes first

Arithmetic operators (a = 7, b = 2)

Operator	Name	Expression	Result	Remark
+	add	a + b	9	also joins strings
-	subtract	a - b	5	
*	multiply	a * b	14	also repeats strings
/	true divide	a / b	3.5	always returns a float
//	floor divide	a // b	3	rounds DOWN to the integer
%	modulus (remainder)	a % b	1	what is left after dividing
**	power	a ** b	49	**0.5 is the square root

```
a, b = 7, 2
print(a / b, a // b, a % b, a ** b)
print(-7 // 2, -7 % 2)          # floor goes toward minus infinity: -4 and 1
print((a // b) * b + a % b == a)  # the identity that always holds -> True
print(divmod(17, 5))             # (quotient, remainder) in one call
print(9 ** 0.5)                  # square root
```

output:

```
3.5 3 1 49
-4 1
True
(3, 2)
3.0
```

Trick:

- n % 2 == 0 → *n is even.* n % 10 → *last digit.* n // 10 → *drop last digit.*
- i % n *wraps an index around a circle of size n (clock, round-robin, circular lists).*

Comparison operators - the answer is always True or False

```
print(5 == 5, 5 != 3, 5 > 3, 5 < 3, 5 >= 5, 5 <= 4)
x = 7
print(1 < x < 10)                # chained comparison - unique to Python
print("apple" < "banana")        # strings compare alphabetically (by Unicode)
print(0.1 + 0.2 == 0.3)          # False! floating point again
```

output:

```
True True True False True False
True
True
False
```

Logical operators

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
age, has_id = 20, True
print(age > 18 and has_id, age < 18 or has_id, not has_id)
# short-circuit: Python stops as soon as the answer is known
x = 0
print(x != 0 and 10 / x > 1)                # False - the division is never executed
# and / or return one of the OPERANDS, not just True/False
print("" or "default")                      # default   (first truthy value)
print(5 and 7)                              # 7         (last value if all truthy)
name = ""
print(name or "Guest")                      # handy way to give a default
```

output:

```
True True False
False
default
7
Guest
```

Assignment, membership, identity, bitwise

```
x = 5
x += 2; print(x)                    # 7   same as x = x + 2  (also -= *= /= //= %= **=)
x **= 2; print(x)                   # 49
print("a" in "cat", 3 not in [1, 2])   # membership
a = [1, 2]; b = [1, 2]; c = a
print(a == b, a is b, a is c)       # equal values / same object / same object
print(5 & 3, 5 | 3, 5 ^ 3, 1 << 3, 16 >> 2)   # bitwise AND OR XOR shifts
```

output:

```
7
49
True True
True False True
1 7 6 8 4
```

Careful:

- = ***puts*** *a value in a variable;* == ***asks*** *whether two values are equal.* if x = 5: *is a SyntaxError.*
- is *checks same object,* == *checks same value. Use* is *only for* None *(*x is None*).*

Operator precedence (highest first)

Level	Operators	Level	Operators
1	( ) brackets	5	== != < > <= >= in not in is
2	** power	6	not
3	+x -x (unary)	7	and
4	* / // % then + -	8	or

```
print(2 + 3 * 4 ** 2)          # 4**2=16 -> 3*16=48 -> 2+48 = 50
print((2 + 3) * 4 ** 2)        # brackets first -> 5*16 = 80
print(-2 ** 2)                 # -4 : power binds tighter than unary minus
print(True or False and False) # and before or -> True or (False) -> True
```

output:

```
50
80
-4
True
```

Trick: Same-level operators run left to right (except **, which runs right to left: 2 ** 3 ** 2 = 2 ** 9 = 512). When in doubt, **add brackets** - clearer to humans too.

Try it yourself:

- Given seconds = 3725, print hours, minutes and seconds using // and %.
- Check if a number is divisible by both 3 and 5 using one expression.
- What is not 1 == 2 and 3 > 2 or False? Work it out with the precedence table, then verify.

6 Conditional Statements

Teaching your program to make decisions

A program that always does the same thing is boring. **Conditionals** let it choose a path depending on whether a condition is True.

Keyword	Meaning	How many allowed
if cond:	run this block if cond is True	exactly one, always first
elif cond:	else-if: tried only if all earlier conditions were False	zero or more
else:	run if none of the above were True	zero or one, always last

- Python uses **indentation (4 spaces)** instead of { } to show what belongs inside.
- Do not forget the **colon :** at the end of if, elif and else lines.
- The condition can be any expression - Python tests its truthiness.

```
marks = 72
if marks >= 80:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

output:
```
Grade B
```

Only the FIRST true branch runs - so order matters

```
marks = 95
if marks >= 60:
    print("Grade B")      # runs first - WRONG answer for 95
elif marks >= 80:
    print("Grade A")      # can never be reached
```

output:
```
Grade B
```

Careful: Always test the **most specific / strictest** condition first (>= 80 before >= 60). Separate if statements (without elif) are independent - **all** true ones run.

```
n = 15
if n % 3 == 0: print("Fizz")      # independent ifs: both can run
if n % 5 == 0: print("Buzz")
```

output:
```
Fizz
Buzz
```

Nested if vs combined conditions

```
age, has_ticket = 20, True
if age >= 18:                          # nested
    if has_ticket:
        print("Enter")
if age >= 18 and has_ticket:           # same thing, flatter and easier to read
    print("Enter")
status = "Pass" if 72 >= 50 else "Fail"   # one-line (ternary) if
print(status)
```

output:

```
Enter
Enter
Pass
```

Useful condition patterns

```
name, items, score = "Ali", [], 87
if name:            print("name is not empty")          # truthiness
if not items:       print("list is empty")
if 0 <= score <= 100:  print("valid score")              # chained comparison
if "x" in "xyz":    print("found x")                     # membership
for year in (2024, 1900, 2000):
    leap = year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
    print(year, "leap" if leap else "not leap")
```

output:

```
name is not empty
list is empty
valid score
found x
2024 leap
1900 not leap
2000 leap
```

match - case (Python 3.10+): a tidy alternative to long elif chains

```
day = 3
match day:
    case 1: print("Mon")
    case 2: print("Tue")
    case 3 | 4: print("Mid-week")     # | means "or"
    case _: print("Other")            # _ is the default
```

output:

```
Mid-week
```

pass - an empty block placeholder

```
if True:
    pass        # a block cannot be empty; pass means "do nothing yet"
print("ok")
```

output:

```
ok
```

Key idea: *Decision recipe: (1) write the rule in plain English, (2) list the possible outcomes, (3) put the narrowest condition first, (4) finish with* `else` *for "everything else".*

Try it yourself:

- *Read a number and print whether it is positive, negative or zero.*
- *BMI calculator: <18.5 underweight, <25 normal, <30 overweight, else obese.*
- *Write a simple login check: correct username AND password → welcome; correct user wrong password → hint.*

7 Loops

Repeat work without repeating code

	for loop	while loop
Use when	you know the items / number of repeats	you repeat until a condition changes
Example	each fruit in a list, numbers 1 to 10	keep asking until the password is right
Danger	very low	infinite loop if the condition never becomes False

for loop and range()

Call	Produces	Rule
range(5)	0 1 2 3 4	start=0, stop excluded
range(1, 6)	1 2 3 4 5	start included, stop excluded
range(0, 10, 2)	0 2 4 6 8	third number = step
range(5, 0, -1)	5 4 3 2 1	negative step counts down

```
for i in range(1, 6):
    print(i, end=" ")
print()
fruits = ["apple", "mango", "kiwi"]
for f in fruits:                          # loop straight over the items
    print(f)
for i, f in enumerate(fruits, start=1):   # item AND its number
    print(i, f)
for name, mark in zip(["Ali", "Sara"], [80, 92]):   # two lists in step
    print(name, mark)
for ch in "abc":                          # a string is a sequence too
    print(ch, end="|")
```

output:

```
1 2 3 4 5
apple
mango
kiwi
1 apple
2 mango
3 kiwi
Ali 80
Sara 92
a|b|c|
```

while loop – three parts you must never forget

*(1) **initialise** the counter, (2) write the **condition**, (3) **update** something inside so the condition can become False.*

```
count = 1           # 1. start
while count <= 3:   # 2. condition
    print(count)
    count += 1      # 3. update - without it the loop never ends!
```

```
output:
1
2
3
```

break and continue

- break - leave the loop completely.
- continue - skip the rest of this round and jump to the next one.

```
for i in range(1, 6):
    if i == 3:
        continue          # skip 3
    if i == 5:
        break             # stop at 5
    print(i)
```

```
output:
1
2
4
```

i	i == 3 ?	i == 5 ?	prints?
1	no	no	1
2	no	no	2
3	yes → continue	(skipped)	nothing
4	no	no	4
5	no	yes → break	loop ends

while True + break: loop with an exit door

```
total = 0
n = 0
while True:
    n += 1
    total += n
    if total > 20:      # exit door
        break
print(n, total)         # 6 21
```

```
output:
6 21
```

for ... else - the else runs only if the loop did NOT break

```
for n in (7, 8):
    for d in range(2, n):
        if n % d == 0:
            print(n, "is not prime")
            break
    else:
        print(n, "is prime")
```

output:

```
7 is prime
8 is not prime
```

Nested loops

*The inner loop runs completely for **every** step of the outer loop: 3 x 4 = 12 repetitions in total.*

```
for i in range(1, 3):
    for j in range(1, 4):
        print(i, "x", j, "=", i * j)
for row in range(1, 5):          # star triangle
    print("*" * row)
```

output:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
*
**
***
****
```

Patterns you will write again and again

```
nums = [4, 9, 2, 7]
total = 0                        # accumulator
for n in nums:
    total += n
print(total, total / len(nums))  # 22 5.5
biggest = nums[0]                # find maximum
for n in nums:
    if n > biggest:
        biggest = n
print(biggest)
fact = 1                         # factorial
for k in range(1, 6):
    fact *= k
print(fact)                      # 120
```

output:

```
22 5.5
9
120
```

Careful:

- ***Off-by-one:*** range(1, 5) *stops at 4. For 1..5 write* range(1, 6).
- *Do not add/remove items of a list **while looping over it** - loop over a copy (*for x in items[:]*) or build a new list.*
- *A* while *whose variable is never updated runs forever - stop it with* Ctrl + C.

Try it yourself:

- *Print the multiplication table of any number typed by the user.*
- *Sum all even numbers from 1 to 100 (answer 2550).*
- *Print an inverted star triangle and a number pyramid.*
- *Guessing game: random secret number, loop until the user guesses it.*

8 Functions

Name a block of code once, reuse it everywhere

A **function** is a named, reusable block of code. Benefits: no copy-paste (DRY - Don't Repeat Yourself), easier testing, and readable programs built from small pieces.

Part	Role	In def add(a, b):
def	keyword that starts a definition	def
name	how you call it later (snake_case)	add
parameters	placeholder names in the definition	a, b
arguments	actual values you pass when calling	add(3, 4) → 3 and 4
body	indented code that runs	...
return	sends a value back and ends the function	return a + b

```
def greet(name):
    print("Hello", name)      # prints, but returns None

def add(a, b):
    return a + b              # sends the result back

greet("Ali")
result = add(3, 4)
print(result)
print(greet("Sara"))          # None - greet has no return
```

output:
```
Hello Ali
7
Hello Sara
None
```

Key idea: print **shows** a value to the human; return **gives** a value to the program. A function that only prints cannot be used inside further calculations.

Returning several values

```
def min_max(nums):
    return min(nums), max(nums)      # really returns ONE tuple

lo, hi = min_max([4, 9, 2, 7])       # unpacking
print(lo, hi)
```

output:
```
2 9
```

Default and keyword arguments

```
def power(base, exp=2):          # exp is 2 when not given
    return base ** exp

print(power(5))                  # 25
print(power(2, 3))               # 8  (positional)
print(power(exp=3, base=2))      # 8  (keyword - order does not matter)
```

output:

```
25
8
8
```

Careful: *Parameters with defaults must come* ***after*** *those without:* `def f(a, b=2)` *is fine,* `def f(a=1, b)` *is a SyntaxError.*

The mutable-default trap

```
def add_item(x, bag=[]):         # BAD: the list is created once, at definition
    bag.append(x)
    return bag
print(add_item(1), add_item(2))  # surprise: the same list is reused

def add_item_ok(x, bag=None):    # GOOD pattern
    if bag is None:
        bag = []
    bag.append(x)
    return bag
print(add_item_ok(1), add_item_ok(2))
```

output:

```
[1, 2] [1, 2]
[1] [2]
```

*args and **kwargs

```
def total(*nums):                # any number of positional args -> tuple
    return sum(nums)
print(total(1, 2, 3, 4))

def info(**data):                # any number of name=value pairs -> dict
    print(data)
info(name="Ali", age=20)

def show(a, b=0, *args, **kwargs):   # required order of kinds of parameters
    print(a, b, args, kwargs)
show(1, 2, 3, 4, x=5)

values = [3, 4]
print(pow(*values))              # *list unpacks a list into separate arguments
```

output:

```
10
{'name': 'Ali', 'age': 20}
1 2 (3, 4) {'x': 5}
81
```

Lambda - a tiny anonymous function

```
square = lambda x: x * x                          # same as def square(x): return x * x
print(square(6))
people = [("Ali", 22), ("Sara", 19), ("Zed", 25)]
print(sorted(people, key=lambda t: t[1]))         # sort by age
print(list(map(lambda x: x * 2, [1, 2, 3])))      # apply to every item
print(list(filter(lambda x: x % 2 == 0, range(10))))   # keep matching items
```

output:

```
36
[('Sara', 19), ('Ali', 22), ('Zed', 25)]
[2, 4, 6]
[0, 2, 4, 6, 8]
```

Scope - where a variable lives (LEGB rule)

Letter	Scope	Meaning
L	Local	names created inside the current function
E	Enclosing	names in an outer function (for nested functions)
G	Global	names at the top level of the file
B	Built-in	print, len, range ... provided by Python

```
x = 10                  # global
def f():
    x = 99              # a NEW local x - the global is untouched
    return x
print(f(), x)

counter = 0
def bump():
    global counter      # say you really mean the global one
    counter += 1
bump(); bump()
print(counter)
```

output:

```
99 10
2
```

Why it works: Python looks for a name in Local → Enclosing → Global → Built-in and stops at the first match. Prefer **passing arguments and returning results** over global - it keeps functions predictable and testable.

Recursion - a function that calls itself

```
def factorial(n):
    if n <= 1:                          # base case: stops the recursion
        return 1
    return n * factorial(n - 1)         # smaller problem

def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(factorial(5), [fib(i) for i in range(8)])
```

output:

```
120 [0, 1, 1, 2, 3, 5, 8, 13]
```

- *Every recursive function **needs a base case**, otherwise it recurses until Python raises `RecursionError` (default limit about 1000 calls).*
- *Functions are objects: you can pass them to other functions, e.g. `apply(f, x)`.*
- *Type hints (`def add(a: int, b: int) -> int:`) document intent; Python does not enforce them.*

Try it yourself:

- *Write `is_prime(n)` and use it to print all primes below 50.*
- *Write `average(*nums)` that returns 0 when called with nothing.*
- *Write a recursive `sum_digits(n)`.*

9 List

The workhorse container: ordered, changeable, anything goes

A **list** is an ordered collection written with [ ]. It is **mutable** (can change), allows duplicates, can hold mixed types and grows or shrinks as needed. Indexing and slicing work exactly like strings, and lists can be changed through them.

```
nums = [10, 20, 30, 40]
print(nums[0], nums[-1], nums[1:3])
nums[0] = 99                        # change a value
nums[1:3] = [21, 31, 32]            # replace a slice (length may change)
print(nums, len(nums))
print(list("abc"), list(range(3)), [0] * 4)    # other ways to create
```

output:

```
10 40 [20, 30]
[99, 21, 31, 32, 40] 5
['a', 'b', 'c'] [0, 1, 2] [0, 0, 0, 0]
```

Methods you need

Method	Effect	Returns
append(x)	add x at the end	None
extend(iterable)	add every item of the iterable	None
insert(i, x)	insert x at position i	None
remove(x)	delete the **first** x (ValueError if missing)	None
pop() / pop(i)	remove and **return** last / i-th item	the item
del lst[i]	delete by position (also slices)	-
index(x) / count(x)	position of first x / how many x	int
sort() / reverse()	in-place; sort(reverse=True, key=...)	None
copy() / clear()	shallow copy / remove everything	list / None

```
nums = [10, 20, 30, 40]
nums.append(50); nums.insert(1, 15); nums.remove(30)
last = nums.pop()
print(nums, last)
nums.sort(); print(nums)
nums.sort(reverse=True); print(nums)
print(len(nums), sum(nums), max(nums), min(nums))
a = [1, 2]
a.append([3, 4]);  print(a)          # append adds ONE item (a list)
b = [1, 2]
b.extend([3, 4]);  print(b)          # extend adds each item
print([1, 2] + [3, 4])               # + creates a new list
```

output:

```
[10, 15, 20, 40] 50
[10, 15, 20, 40]
[40, 20, 15, 10]
4 85 40 10
```

```
[1, 2, [3, 4]]
[1, 2, 3, 4]
[1, 2, 3, 4]
```

Careful: nums = nums.sort() *sets* nums *to* ***None*** *-* sort() *changes the list in place and returns nothing. Use* sorted(nums) *when you want a new sorted list and the original untouched.*

Aliasing vs copying - the most important list idea

```
a = [1, 2, 3]
b = a                                   # NOT a copy: two names, one list
b.append(4)
print(a)                                # [1, 2, 3, 4] - a changed too!
c = a.copy()                            # real (shallow) copy; also a[:] or list(a)
c.append(5)
print(a, c)
import copy
grid = [[1, 2], [3, 4]]
shallow = grid.copy(); deep = copy.deepcopy(grid)
grid[0][0] = 99
print(shallow[0][0], deep[0][0])        # 99 1 - shallow copy shares inner lists
```

output:

```
[1, 2, 3, 4]
[1, 2, 3, 4] [1, 2, 3, 4, 5]
99 1
```

Nested lists (2-D data, matrices)

```
matrix = [[1, 2, 3],
          [4, 5, 6]]
print(matrix[1][2], len(matrix), len(matrix[0]))    # row, then column
for row in matrix:
    print(row, sum(row))
bad = [[0] * 3] * 2                     # BUG: two references to the SAME row
bad[0][0] = 1
print(bad)
good = [[0] * 3 for _ in range(2)]      # correct: a new row each time
good[0][0] = 1
print(good)
```

output:

```
6 2 3
[1, 2, 3] 6
[4, 5, 6] 15
[[1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0]]
```

List comprehension - build a list in one line

Pattern: [expression for item in iterable if condition]. Read it as "give me expression for each item that passes the condition".

```
squares = [x * x for x in range(1, 6)]
even = [x for x in range(10) if x % 2 == 0]
print(squares, even)
# the long way, for comparison:
squares2 = []
for x in range(1, 6):
    squares2.append(x * x)
print(squares == squares2)
print([x if x > 0 else 0 for x in [-2, 3, -1, 5]])      # if-else goes BEFORE for
print([(i, j) for i in range(2) for j in range(2)])      # nested loops
words = ["hi", "python", "ok"]
print([w.upper() for w in words if len(w) > 2])
```

output:

```
[1, 4, 9, 16, 25] [0, 2, 4, 6, 8]
True
[0, 3, 0, 5]
[(0, 0), (0, 1), (1, 0), (1, 1)]
['PYTHON']
```

Handy built-ins for lists

```
marks = [55, 80, 67, 92]
print(sorted(marks), list(reversed(marks)))
print(any(m > 90 for m in marks), all(m >= 50 for m in marks))
print(80 in marks, marks.index(67), marks.count(55))
stack = []                                              # a list as a stack (last in, first out)
stack.append(1); stack.append(2); print(stack.pop())
```

output:

```
[55, 67, 80, 92] [92, 67, 80, 55]
True True
True 2 1
2
```

Try it yourself:

- *Remove duplicates from a list while keeping the first-seen order.*
- *Given a list of marks, build a new list containing only marks above the average.*
- *Transpose a 2x3 matrix into 3x2 using nested loops, then using a comprehension.*

10 Array (and NumPy)

Fast numbers: the data structure behind all of machine learning

*An **array** stores items of the **same type** next to each other in memory, so maths on it is fast. Python gives you three options:*

	list	array.array	**NumPy** ndarray
Item types	*mixed*	*one C type*	*one type (dtype)*
Dimensions	*nested lists*	*1-D only*	*any number (1-D, 2-D, 3-D ...)*
Maths on all items	*write a loop*	*write a loop*	***one expression** (vectorised)*
Speed for numbers	*slow*	*medium*	*very fast (C loops)*
Used in ML?	*for small things*	*almost never*	***everywhere***

> ***Note-fix (more precise than the original):** The original notes say Python has no built-in array. Precisely: Python has the small standard `array` module, but it is rarely used - lists act as arrays in everyday code, and **NumPy** is the real array tool for AI/ML.*

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr * 2, arr + 10, arr ** 2)          # whole-array maths, no loop
print(arr.mean(), arr.sum(), arr.std())
print(arr.shape, arr.ndim, arr.size, arr.dtype)
print([1, 2] * 2, np.array([1, 2]) * 2)     # list repeats, array multiplies
```

output:

```
[2 4 6 8] [11 12 13 14] [ 1  4  9 16]
2.5 10 1.1180339887498895
(4,) 1 4 int64
[1, 2, 1, 2] [2 4]
```

Creating arrays

```
import numpy as np
print(np.zeros(3), np.ones((2, 2)))
print(np.arange(0, 10, 3), np.linspace(0, 1, 5))
print(np.eye(2))                            # identity matrix
print(np.array([1, 2.5]).dtype, np.array([1, 2]).dtype)   # all items share one type
```

output:

```
[0. 0. 0.] [[1. 1.]
 [1. 1.]]
[0 3 6 9] [0.   0.25 0.5  0.75 1.  ]
[[1. 0.]
 [0. 1.]]
float64 int64
```

2-D arrays: indexing, slicing, reshaping

```
import numpy as np
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.shape)                      # (2, 3) = 2 rows, 3 columns
print(m[1, 2], m[0], m[:, 0])       # element, first row, first column
print(m.T)                          # transpose
print(np.arange(6).reshape(2, 3))
print(m.sum(axis=0), m.sum(axis=1)) # down the columns / across the rows
```

output:

```
(2, 3)
6 [1 2 3] [1 4]
[[1 4]
 [2 5]
 [3 6]]
[[0 1 2]
 [3 4 5]]
[5 7 9] [ 6 15]
```

Boolean masks - filter with a condition

```
import numpy as np
scores = np.array([45, 82, 67, 91, 30])
print(scores > 60)                  # array of True/False
print(scores[scores > 60])          # keep only the True positions
scores[scores < 50] = 50            # change them in place
print(scores, scores.argmax())      # index of the biggest
```

output:

```
[False  True  True  True False]
[82 67 91]
[50 82 67 91 50] 3
```

Broadcasting and the dot product

```
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6]])
print(m + np.array([10, 20, 30]))          # the 1-D row is stretched across every row
x = np.array([1, 2, 3]); w = np.array([0.5, 0.1, 2.0])
print(np.dot(x, w), x @ w)                 # 1*0.5 + 2*0.1 + 3*2.0 = 6.7
A = np.array([[1, 2], [3, 4]]); B = np.array([[0, 1], [1, 0]])
print(A @ B)                               # matrix multiplication
```

output:

```
[[11 22 33]
 [14 25 36]]
6.7 6.7
[[2 1]
 [4 3]]
```

Why it works: *A prediction in ML is almost always a* ***dot product****: prediction = w · x + b. NumPy computes it for thousands of rows at once, which is why linear regression, neural networks and every library you will use are built on NumPy arrays.*

Try it yourself:

- *Normalise* `[10, 20, 30, 40]` *to the 0-1 range:* `(x - min) / (max - min)`.
- *Create a 3x3 array 1..9 and print its diagonal sum (*`np.trace`*) and each row's mean.*
- *Compute the distance between two points with* `np.sqrt(((p - q) ** 2).sum())`.

11 Tuple

A list that promises never to change

*A **tuple** is an ordered collection written with ( ). It is **immutable**: once created, you cannot add, remove or replace items. That makes it safe for fixed data (coordinates, RGB colours, dates) and slightly faster and lighter than a list.*

```
point = (3, 4)
print(point[0], point[-1], point[0:1], len(point))
x, y = point                          # unpacking: one name per item
print(x, y)
single = (5,)                         # ONE item needs a trailing comma
print(type(single), type((5)))        # (5) is just 5 in brackets!
empty = ()
nums = tuple([1, 2, 2, 3])            # convert from a list
print(nums.count(2), nums.index(3))
try:
    point[0] = 9
except TypeError as e:
    print("TypeError:", e)
```

output:

```
3 4 (3,) 2
3 4
<class 'tuple'> <class 'int'>
2 3
TypeError: 'tuple' object does not support item assignment
```

Packing, unpacking, swapping

```
coords = 10, 20, 30                   # brackets are optional: this is a tuple (packing)
a, b, c = coords                      # unpacking
first, *rest = (1, 2, 3, 4)           # star takes "everything else"
print(first, rest)
a, b = b, a                           # the swap trick is tuple packing + unpacking
print(a, b)
def stats(nums):                      # functions "return several values" as a tuple
    return min(nums), max(nums), sum(nums) / len(nums)
print(stats([2, 4, 9]))
```

output:

```
1 [2, 3, 4]
20 10
(2, 9, 5.0)
```

Immutability is shallow

```
t = (1, [10, 20], "x")
t[1].append(30)                    # allowed: the tuple still points at the same list
print(t)
locations = {(33.7, 73.0): "Islamabad"}   # tuples can be dict keys (lists cannot)
print(locations[(33.7, 73.0)])
from collections import namedtuple
Point = namedtuple("Point", "x y")         # a tuple with named fields
p = Point(3, 4)
print(p.x, p[1], p)
```

output:

```
(1, [10, 20, 30], 'x')
Islamabad
3 4 Point(x=3, y=4)
```

	List [ ]	Tuple ( )
Changeable	yes	no
Methods	many (append, remove, sort ...)	only count, index
Hashable (usable as dict key / set item)	no	yes (if all items are hashable)
Typical use	collection that grows / changes	fixed record, return values, keys

Key idea: Use a **list** for a collection of similar things that change; use a **tuple** for a record of different things that stays fixed (name, age, city).

Try it yourself:

- Write a function returning (quotient, remainder) and unpack the result.
- Why does (1, 2) + (3,) work although tuples are immutable? (Hint: it builds a new tuple.)

12 Dictionary

Look things up by name, not by position

*A **dictionary** (dict) stores **key : value** pairs inside { }. You find a value by its key, and the lookup is very fast (it uses hashing, so it does not matter whether the dict has 10 or 10 million items). Since Python 3.7 a dict also remembers insertion order.*

```
student = {"name": "Ali", "age": 20, "city": "Lahore"}
print(student["name"])
print(student.get("marks", 0))          # safe: 0 if the key is missing
student["age"] = 21                     # change
student["marks"] = 85                   # add a new pair
del student["city"]                     # delete
print(student, len(student))
print("name" in student, "Ali" in student)   # `in` checks KEYS only
try:
    student["height"]
except KeyError as e:
    print("KeyError:", e)
```

output:

```
Ali
0
{'name': 'Ali', 'age': 21, 'marks': 85} 3
True False
KeyError: 'height'
```

Looping and views

```
student = {"name": "Ali", "age": 21, "marks": 85}
print(list(student.keys()))
print(list(student.values()))
for key, value in student.items():      # the standard way to loop
    print(key, "->", value)
print(student.pop("marks"), student)    # remove and return a value
student.update({"age": 22, "city": "Lahore"})   # merge / overwrite several at once
print(student.setdefault("country", "PK"), student)
```

output:

```
['name', 'age', 'marks']
['Ali', 21, 85]
name -> Ali
age -> 21
marks -> 85
85 {'name': 'Ali', 'age': 21}
PK {'name': 'Ali', 'age': 22, 'city': 'Lahore', 'country': 'PK'}
```

Operation	Code	Notes
create	{} dict(a=1, b=2) dict(zip(keys, values))	{} is an empty **dict**, not a set
read	d[k] d.get(k, default)	[] raises KeyError, get does not
write	d[k] = v d.update(other) d.setdefault(k, v)	same syntax adds or replaces
delete	del d[k] d.pop(k) d.popitem() d.clear()	pop returns the value
iterate	for k in d d.keys() d.values() d.items()	for k in d gives keys

Real examples: counting, nesting, comprehension, sorting

```
text = "to be or not to be"
count = {}
for word in text.split():
    count[word] = count.get(word, 0) + 1                    # the classic counting pattern
print(count)
print(sorted(count.items(), key=lambda kv: kv[1], reverse=True)[:2])    # top 2
squares = {n: n * n for n in range(1, 5)}                   # dict comprehension
print(squares)
students = {
    "ali":  {"age": 20, "marks": [80, 90]},
    "sara": {"age": 19, "marks": [95, 85]},
}
print(students["sara"]["marks"][1])                         # key, key, index
for name, info in students.items():
    print(name, sum(info["marks"]) / len(info["marks"]))
```

output:

```
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
[('to', 2), ('be', 2)]
{1: 1, 2: 4, 3: 9, 4: 16}
85
ali 85.0
sara 90.0
```

Careful:

- Keys must be **hashable** (immutable): strings, numbers, tuples are fine; a list as key raises TypeError: unhashable type.
- Keys are unique - assigning to an existing key **overwrites** it silently.
- Do not change a dict's size while looping over it; loop over list(d) if you must delete.

Why it works: Finding an item in a list means checking items one by one (time grows with size). A dict jumps straight to the item using the key's hash - use a dict whenever you search often (ids, names, caches, counting).

Try it yourself:

- Count how often each letter appears in a sentence and print the three most frequent.
- Invert a dict ({"a": 1} → {1: "a"}) with a comprehension.
- Store 3 students with their marks and print the one with the highest average.

13 Set

Unique items only - and set maths for free

*A **set** is an unordered collection of **unique** items written with { }. There is no index and no duplicates. Sets are perfect for removing duplicates, testing membership very quickly, and for union / intersection style questions.*

```
s = {1, 2, 2, 3}
print(s, len(s))                    # duplicates vanish
print(set([3, 1, 3, 2]), set("hello"))   # build from any iterable
empty = set()                       # {} would make an empty DICT
print(type(empty), type({}))
s.add(10); s.add(2)                 # adding an existing item changes nothing
s.discard(99)                       # no error if missing...
try:
    s.remove(99)                    # ...but remove raises KeyError
except KeyError as e:
    print("KeyError:", e)
print(2 in s)                       # very fast membership test
```

output:

```
{1, 2, 3} 3
{1, 2, 3} {'e', 'l', 'h', 'o'}
<class 'set'> <class 'dict'>
KeyError: 99
True
```

Set operations

Operation	Symbol	Method	{1,2,3} and {3,4}
union (everything)	a | b	a.union(b)	{1, 2, 3, 4}
intersection (in both)	a & b	a.intersection(b)	{3}
difference (only in a)	a - b	a.difference(b)	{1, 2}
symmetric difference (in exactly one)	a ^ b	a.symmetric_difference(b)	{1, 2, 4}
subset / superset	a <= b a >= b	issubset issuperset	False False
no common items?	-	a.isdisjoint(b)	False

```
a = {1, 2, 3}; b = {3, 4}
print(a | b, a & b, a - b, a ^ b)
print({1, 2} <= a, a.isdisjoint({9}))
emails = ["a@x.com", "b@x.com", "a@x.com", "c@x.com", "b@x.com"]
print(len(set(emails)), "unique addresses")
unique_sorted = sorted(set([5, 3, 5, 1, 3]))      # sets lose order; sort if you need one
print(unique_sorted)
class_a = {"Ali", "Sara", "Zed"}; class_b = {"Sara", "Omar"}
print("in both:", class_a & class_b)
print({x * x for x in range(5)})                  # set comprehension
f = frozenset([1, 2])                             # immutable set (can be a dict key)
print(f)
```

output:

```
{1, 2, 3, 4} {3} {1, 2} {1, 2, 4}
True True
3 unique addresses
[1, 3, 5]
in both: {'Sara'}
{0, 1, 4, 9, 16}
frozenset({1, 2})
```

Careful: *Set items must be hashable (no lists inside a set), and* ***order is not guaranteed*** *- never rely on the printed order.*

Which container should I use?

	List [ ]	**Tuple** ( )	**Dict** {k: v}	**Set** { }
Ordered	*yes*	*yes*	*yes (insertion)*	*no*
Changeable	*yes*	*no*	*yes*	*yes (items are not editable)*
Duplicates	*allowed*	*allowed*	*keys unique*	*never*
Access by	*index*	*index*	*key*	*membership only*
Choose it when...	*ordered collection that changes*	*fixed record*	*lookup by name / id*	*uniqueness, set maths*

Try it yourself:

- *Which students attend both clubs? Which attend exactly one?*
- *Find the first repeated number in* `[4, 2, 7, 2, 9]` *using a set to remember what you have seen.*
- *Check whether two words are anagrams using sets (then think why sets alone are not enough for "aab" and "abb").*

14 File Handling

Make your data survive after the program ends

Variables vanish when a program stops. **Files** store data permanently. The core function is open(filename, mode), and the safe way to use it is the with statement, which **closes the file automatically** - even if an error occurs.

Mode	Meaning	If the file exists	If it does not exist
"r"	read (default)	reads it	FileNotFoundError
"w"	write	**ERASES old content**	creates it
"a"	append (add at the end)	keeps content, adds after it	creates it
"x"	create new	FileExistsError	creates it
"r+"	read and write	keeps content	FileNotFoundError
add b	binary ("rb", "wb") for images, PDFs	-	-

Writing

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Hello Python\n")                    # \n = new line - write() does not add one
    f.write("File handling is easy\n")
    f.writelines(["line three\n", "line four\n"])    # a list of strings
print("written")
```

output:

```
written
```

Reading - four ways

```
with open("notes.txt") as f:
    text = f.read()                 # whole file as ONE string
print(repr(text))
with open("notes.txt") as f:
    print(repr(f.readline()))       # just the next line (including \n)
with open("notes.txt") as f:
    lines = f.readlines()           # list of lines
print(len(lines))
with open("notes.txt") as f:
    for line in f:                  # best for big files: one line at a time
        print(line.strip())         # strip() removes the \n
```

output:

```
'Hello Python\nFile handling is easy\nline three\nline four\n'
'Hello Python\n'
4
Hello Python
File handling is easy
line three
line four
```

Appending, checking, deleting

```
with open("notes.txt", "a") as f:
    f.write("A new line added\n")
import os
print(os.path.exists("notes.txt"), os.path.getsize("notes.txt"), "bytes")
with open("notes.txt") as f:
    print(sum(1 for _ in f), "lines now")
```

output:

```
True 73 bytes
5 lines now
```

Handling errors: try / except / else / finally

```
try:
    with open("missing.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("File not found!")
except PermissionError:
    print("No permission")
else:
    print("only runs if NO error happened")
finally:
    print("Always runs")
```

output:

```
File not found!
Always runs
```

Key idea: try - *code that might fail*. except - *what to do if it fails*. else - *runs only on success*. finally - *runs no matter what (clean-up)*.

A realistic mini-task: numbers file → average

```
with open("numbers.txt", "w") as f:
    f.write("10\n20\n x \n30\n")
values = []
with open("numbers.txt") as f:
    for line in f:
        try:
            values.append(float(line))                    # float() ignores surrounding spaces
        except ValueError:
            print("skipped bad line:", repr(line.strip()))
print(values, sum(values) / len(values))
```

output:

```
skipped bad line: 'x'
[10.0, 20.0, 30.0] 20.0
```

```
import csv, os
with open("marks.csv", "w", newline="") as f:
```

```
    w = csv.writer(f)
    w.writerow(["name", "marks"]); w.writerow(["Ali", 80]); w.writerow(["Sara",
92])
with open("marks.csv") as f:
    for row in csv.DictReader(f):                                    # each row becomes a dict
        print(row["name"], int(row["marks"]) + 1)
os.remove("notes.txt"); os.remove("numbers.txt"); os.remove("marks.csv")
print(os.path.exists("notes.txt"))
```

output:

```
Ali 81
Sara 93
False
```

Careful:

- *"w" erases the file **the moment you open it** - even if you write nothing. Use "a" to keep old data.*
- *Relative paths ("notes.txt") are relative to the folder the program is **run from**, not where the script lives.*
- *On Windows use raw strings or forward slashes: r"C:\data\f.txt" or "C:/data/f.txt".*
- *Always pass encoding="utf-8" when text may contain non-English characters.*

Try it yourself:

- *Write a program that stores a to-do list in a file: add task, show tasks, delete a task.*
- *Count the words in a text file and write the 5 most common to a second file.*
- *Read a CSV of student marks and print the topper.*

15 OOP: Class and Object

Model real things as bundles of data + behaviour

Object Oriented Programming builds programs out of **objects** that combine data (what the thing knows) and functions (what the thing can do). It keeps big programs organised: a Student, a BankAccount or a Car each looks after its own data.

Term	Meaning	Real-world picture
Class	blueprint / template that describes a kind of thing	architect's house plan
Object (instance)	a real thing created from the class	an actual house built from the plan
Attribute	a variable that belongs to an object / class	colour, number of rooms
Method	a function defined inside a class	open the door, switch on the light
__init__	constructor: runs automatically when an object is created	setting the house up on day one
self	the object that is calling the method (first parameter)	"this very house"

```
class Car:
    def __init__(self, brand, speed):
        self.brand = brand                      # attribute
        self.speed = speed
    def drive(self):                            # method
        print(self.brand, "is driving at", self.speed)
    def __str__(self):                          # what print(obj) shows
        return f"Car({self.brand})"

c1 = Car("Toyota", 80)                          # create objects (instances)
c2 = Car("Honda", 100)
c1.drive()
print(c2)
c1.speed = 120                                  # objects are independent copies of data
print(c1.speed, c2.speed)
```

output:

```
Toyota is driving at 80
Car(Honda)
120 100
```

Why it works: When you write c1.drive(), Python actually calls Car.drive(c1): the object before the dot is passed automatically as self. That is why every method needs self as its first parameter and why attributes are written self.name.

__str__ vs __repr__

```
class P:
    def __init__(self, x): self.x = x
    def __str__(self):  return f"P with x={self.x}"     # friendly, for users / print()
    def __repr__(self): return f"P({self.x})"           # exact, for developers / debugging
pt = P(5)
print(pt)                                               # uses __str__
print([pt, P(6)])                                       # containers show items using __repr__
```

output:
P with x=5
[P(5), P(6)]

Class variable vs instance variable

```
class Student:
    school = "COMSATS"                  # class variable: ONE copy shared by all objects
    count = 0
    def __init__(self, name):
        self.name = name                # instance variable: each object has its own
        Student.count += 1              # update shared data through the class name

s1 = Student("Ali"); s2 = Student("Sara")
print(s1.name, s1.school, Student.count)
Student.school = "NUST"                 # change once -> everyone sees it
print(s2.school)
s1.school = "LUMS"                      # creates an instance variable that hides the class one
print(s1.school, s2.school, Student.school)
```

output:
Ali COMSATS 2
NUST
LUMS NUST NUST

An object that changes its own state

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner, self.balance = owner, balance
    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit must be positive")
        self.balance += amount
    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
            return
        self.balance -= amount

acc = BankAccount("Ali", 100)
acc.deposit(50); acc.withdraw(500); acc.withdraw(30)
print(acc.owner, acc.balance)
```

output:
Insufficient funds
Ali 120

Special ("dunder") methods - make your class behave like built-ins

Method	Called by	Method	Called by
__init__	Car(...)	__eq__	a == b
__str__	print(a), str(a)	__lt__	a < b *(and sorted)*
__repr__	*debugger, lists*	__add__	a + b
__len__	len(a)	__getitem__	a[i]

```
class Car:
    def __init__(self, brand): self.brand = brand
c = Car("Kia")
print(type(5).__name__, type("hi").__name__, type([1]).__name__)      # built-ins are classes too
print(hasattr(c, "brand"), getattr(c, "brand"), c.__dict__)
```

output:

```
int str list
True Kia {'brand': 'Kia'}
```

Careful:

- *Forgetting* self *in* def drive(): → TypeError: drive() takes 0 positional arguments but 1 was given.
- *Writing* brand = brand *instead of* self.brand = brand *in* __init__ *stores nothing on the object.*
- *A class variable that is a* ***list*** *is shared by all objects - put mutable data in* __init__ *instead.*

Try it yourself:

- *Create a* Rectangle(width, height) *class with* area(), perimeter() *and a nice* __str__.
- *Add* __eq__ *so two rectangles with equal area compare equal.*
- *Create a* Student *class that keeps a list of marks and returns the average.*

16 The 4 Pillars of OOP

Encapsulation - Inheritance - Polymorphism - Abstraction

Pillar	One-line meaning	Everyday analogy	Python tools
Encapsulation	bundle data + methods and control access to the data	ATM: you use buttons, not the cash box	_name, __name, @property
Inheritance	a child class reuses and extends a parent	child inherits family traits	class Dog(Animal), super()
Polymorphism	same method name, different behaviour per class	"speak" - dog barks, cat meows	overriding, duck typing, __add__
Abstraction	expose what it does, hide how; force a common interface	car pedals: you do not see the engine	ABC, @abstractmethod

I. Encapsulation - hide and protect data

Style	Meaning	Real effect in Python
name	public	anyone can read / change it
_name	protected (convention)	"please do not touch" - only a hint
__name	private	name mangling: stored as _Class__name, hard (not impossible) to reach from outside

```
class Account:
    def __init__(self, balance):
        self.__balance = balance          # private
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
    def get_balance(self):                # getter
        return self.__balance

acc = Account(100)
acc.deposit(50)
print(acc.get_balance())
try:
    print(acc.__balance)                  # hidden
except AttributeError as e:
    print("AttributeError:", e)
print(acc._Account__balance)              # mangled name - works, but never do this
```

output:

```
150
AttributeError: 'Account' object has no attribute '__balance'
150
```

*The **Pythonic** way to write getters / setters is @property: outside code uses plain attribute syntax, while the class still validates.*

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius                    # goes through the setter below
```

```
    @property
    def celsius(self):
        return self._celsius
    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = value

t = Temperature(25)
t.celsius = 30                          # looks like an attribute, runs the setter
print(t.celsius)
try:
    t.celsius = -500
except ValueError as e:
    print("ValueError:", e)
```

output:

```
30
ValueError: below absolute zero
```

2. Inheritance - the child reuses the parent

*The child gets every attribute and method of the parent, can **add** new ones, and can **override** inherited ones.*
`super()` *calls the parent's version.*

```
class Animal:
    def __init__(self, name):
        self.name = name
    def eat(self):
        print(self.name, "is eating")
    def sound(self):
        print("...")

class Dog(Animal):                                  # Dog inherits Animal
    def __init__(self, name, breed):
        super().__init__(name)                      # let the parent set up name
        self.breed = breed
    def bark(self):
        print("Woof!")
    def sound(self):                                # override
        print("Woof!")

d = Dog("Tommy", "Lab")
d.eat(); d.bark(); d.sound()
print(isinstance(d, Animal), isinstance(d, Dog), issubclass(Dog, Animal))
```

output:

```
Tommy is eating
Woof!
Woof!
True True True
```

Kind	Shape	Syntax
Single	*B inherits A*	class B(A)
Multilevel	*A → B → C (grandparent → parent → child)*	class C(B), class B(A)
Multiple	*C inherits from two parents*	class C(A, B)
Hierarchical	*B and C both inherit A*	class B(A), class C(A)
Hybrid	*any mixture of the above*	-

```
class A:
    def who(self): return "A"
class B(A):
    def who(self): return "B"
class C(A):
    def who(self): return "C"
class D(B, C): pass                          # multiple inheritance (diamond shape)
print(D().who())
print([k.__name__ for k in D.__mro__])       # Method Resolution Order
```

output:

```
B
['D', 'B', 'C', 'A', 'object']
```

Why it works: *With several parents Python searches for a method in the* ***MRO*** *(left to right, children before parents):* D → B → C → A → object. super() *follows that same order. Every class ultimately inherits from* object.

3. Polymorphism - one name, many forms

```
class Cat(Animal):
    def sound(self): print("Meow")
class Cow(Animal):
    def sound(self): print("Moo")

for pet in [Cat("Kitty"), Cow("Bella"), Dog("Rex", "Pug")]:
    pet.sound()                                  # same call, different behaviour
```

output:

```
Meow
Moo
Woof!
```

Python is ***duck typed*** *- if it walks like a duck and quacks like a duck, it is a duck. The loop only needs an object with* sound(), *not a specific parent. Operators can be polymorphic too through special methods:*

```
class Vector:
    def __init__(self, x, y): self.x, self.y = x, y
    def __add__(self, other): return Vector(self.x + other.x, self.y + other.y)
    def __str__(self): return f"({self.x}, {self.y})"
print(Vector(1, 2) + Vector(3, 4))           # + now works for Vector
print(1 + 2, "a" + "b", [1] + [2])           # + is already polymorphic in Python
```

output:

```
(4, 6)
3 ab [1, 2]
```

Note-fix (more precise than the original): Python has **no true method overloading** (two methods with the same name but different parameters). The last definition wins. Use default values or *args instead: def area(self, a, b=None).

4. Abstraction - show only what is needed

An **abstract class** cannot be instantiated; it defines a contract: every child **must** implement the abstract methods.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
    def describe(self):                                    # normal methods are allowed too
        return f"{type(self).__name__} with area {self.area():.2f}"

class Square(Shape):
    def __init__(self, side): self.side = side
    def area(self): return self.side ** 2
class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2

try:
    Shape()
except TypeError as e:
    print("TypeError:", e)
for s in (Square(4), Circle(1)):
    print(s.describe())
```

output:

```
TypeError: Can't instantiate abstract class Shape without an implementation for
  abstract method 'area'
Square with area 16.00
Circle with area 3.14
```

Trick: Encapsulation = protect the data (how is it stored?). **Abstraction** = simplify the interface (what can I do with it?). Students mix them up - think "lock" vs "remote control".

Extra OOP tools

```
class MathTool:
    tools_made = 0
    def __init__(self): MathTool.tools_made += 1
    @staticmethod
    def add(a, b):                                         # no self, no cls: just a function placed in the class
        return a + b
    @classmethod
    def info(cls):                                         # receives the CLASS, not an object
        return f"{cls.__name__}: {cls.tools_made} made"
```

```
    @classmethod
    def from_pair(cls, text):                    # alternative constructor - a very common use
        return cls()

MathTool.from_pair("1,2"); MathTool()
print(MathTool.add(2, 3), MathTool.info())
```

output:
5 MathTool: 2 made

Method type	*First parameter*	*Can use*	*Typical use*
instance method	self	*object data + class data*	*most methods*
@classmethod	cls	*class data only*	*alternative constructors, counters*
@staticmethod	*none*	*neither*	*helper that logically belongs to the class*

Try it yourself:

- *Build* Employee → Manager *(inheritance) with a* salary() *method overridden in* Manager.
- *Create an abstract* Payment *class with* pay(amount) *and implement* CardPayment *and* CashPayment.
- *Give* BankAccount *a private balance with a* @property *that is read-only.*

17 Regression

Teach a computer to predict a number from other numbers

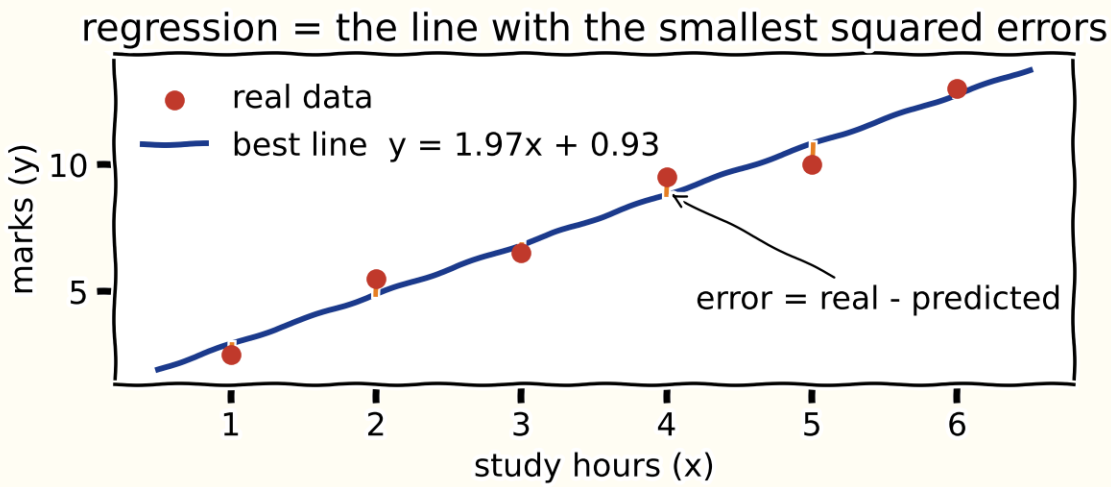
Machine learning in one paragraph

*In **supervised learning** we give the computer examples with the right answer and it learns the pattern. The inputs are called **features** (x: study hours, house size) and the answer is the **target / label** (y: marks, price). After training, the model predicts y for new x it has never seen.*

	Regression	Classification
Predicts	a **number** (continuous)	a **category** (class label)
Examples	house price, temperature, marks	spam / not spam, pass / fail, digit 0-9
Common models	linear regression, ridge, random-forest regressor	logistic regression, decision tree, SVM
Typical error measure	MSE, RMSE, MAE, R^2	accuracy, precision, recall, F1

Simple linear regression: $y = m \cdot x + c$

Symbol	Name	Meaning	Our example (marks vs hours)
x	input / feature	what we know	study hours
y	output / target	what we predict	marks
m	slope (weight, coefficient)	how much y rises when x increases by 1	each extra hour adds m marks
c	intercept (bias)	value of y when x = 0	marks with zero study



What does "best line" mean? - the error

*For one point the **error (residual)** is real - predicted. Positive and negative errors would cancel, so we **square** them. The average of the squared errors is the **MSE** (Mean Squared Error). Regression picks the m and c with the **smallest MSE** - this is called **least squares**.*

Example with noisy data: the best line is y = 1.971x + 0.933.

x (hours)	y (real)	predicted = m·x + c	error = y − pred	squared error
1	2.5	2.905	-0.405	0.164
2	5.5	4.876	+0.624	0.389
3	6.5	6.848	-0.348	0.121
4	9.5	8.819	+0.681	0.464
5	10.0	10.790	-0.790	0.625
6	13.0	12.762	+0.238	0.057
			MSE =	**0.3032**

By hand: the least-squares formulas

$m = \Sigma (x - \bar{x})(y - \bar{y}) / \Sigma (x - \bar{x})^2$ and $c = \bar{y} - m \cdot \bar{x}$ ($\bar{x}$ and $\bar{y}$ are the means).

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1	3	-2	-4	8	4
2	5	-1	-2	2	1
3	7	+0	+0	0	0
4	9	+1	+2	2	1
5	11	+2	+4	8	4
$\bar{x} = 3$	$\bar{y} = 7$			$\Sigma = 20$	$\Sigma = 10$

So $m = 20 / 10 = 2$ and $c = 7 - 2 \times 3 = 1 \rightarrow$ **$y = 2x + 1$**, exactly the rule hidden in the data.

The 5 steps of every regression project

1. **Collect** data (x and y) and look at it (plot it!).
2. **Split** into a train set (to learn) and a test set (to check honestly).
3. **Train / fit** the model on the training data.
4. **Predict** on the test data.
5. **Evaluate** the error (MSE, RMSE, R^2) - and improve.

scikit-learn: the short way

```
from sklearn.linear_model import LinearRegression
hours = [[1], [2], [3], [4], [5]]          # x must be 2-D: rows = samples, columns = features
marks = [3, 5, 7, 9, 11]                   # y is 1-D
model = LinearRegression()
model.fit(hours, marks)                    # learn m and c
print(model.coef_, model.intercept_)       # slope m, intercept c
print(model.predict([[6], [10]]))          # marks for 6 and 10 hours
```

output:

```
[2.] 1.0
[13. 21.]
```

Same thing without a library (to see the maths)

```
xs = [1, 2, 3, 4, 5]
ys = [3, 5, 7, 9, 11]
n = len(xs)
mean_x = sum(xs) / n
mean_y = sum(ys) / n
top = bottom = 0
for x, y in zip(xs, ys):
    top += (x - mean_x) * (y - mean_y)
    bottom += (x - mean_x) ** 2
m = top / bottom                    # slope
c = mean_y - m * mean_x             # intercept
print(m, c)
```

output:

```
2.0 1.0
```

Measuring how good the model is

Metric	Formula (n points)	Reading it
MSE	$(1/n)\ \Sigma\ (y - \hat{y})^2$	*smaller is better; units are y^2 (hard to picture)*
RMSE	$\sqrt{\text{MSE}}$	*typical error in the **same units as y***
MAE	$(1/n)\ \Sigma\ \lvert y - \hat{y}\rvert$	*average absolute error; less sensitive to outliers*
R^2	1 - SS_res / SS_tot	*1 = perfect, 0 = no better than always guessing the mean, **can be negative** on test data*

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
hours = [[1], [2], [3], [4], [5]]
marks = [3, 5, 7, 9, 11]
model = LinearRegression().fit(hours, marks)
pred = model.predict(hours)
print(mean_squared_error(marks, pred), r2_score(marks, pred))      # perfect fit
```

output:

```
0.0 1.0
```

A complete mini-project: train / test split

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score

rng = np.random.default_rng(42)
X = rng.uniform(0, 10, size=(100, 1))                    # 100 students: hours studied
y = 5 + 3 * X[:, 0] + rng.normal(0, 2, size=100)         # true rule 5 + 3x plus random noise

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
model = LinearRegression().fit(X_train, y_train)
pred = model.predict(X_test)

print(f"learned: y = {model.coef_[0]:.2f}x + {model.intercept_:.2f}  (true:
3x + 5)")
print(f"train size {len(X_train)}, test size {len(X_test)}")
print(f"MSE  {mean_squared_error(y_test, pred):.3f}")
print(f"RMSE {mean_squared_error(y_test, pred) ** 0.5:.3f}")
print(f"MAE  {mean_absolute_error(y_test, pred):.3f}")
print(f"R2   {r2_score(y_test, pred):.3f}")
```

output:

```
learned: y = 3.05x + 4.90  (true: 3x + 5)
train size 80, test size 20
MSE  3.435
RMSE 1.853
MAE  1.436
R2   0.952
```

Why it works: *The data has random noise of standard deviation 2, so even a perfect model cannot get an RMSE far below about 2 - that noise is* ***irreducible error****. We evaluate on the* ***test set*** *because a model can look great on data it has already seen (see overfitting below).*

Multiple regression and logistic regression

```
from sklearn.linear_model import LinearRegression, LogisticRegression
# y = m1*x1 + m2*x2 + c  (hours studied, hours slept)  ->  marks
X = [[1, 8], [2, 6], [3, 7], [4, 5], [5, 8], [6, 6]]
y = [35, 44, 55, 62, 78, 82]
mr = LinearRegression().fit(X, y)
print("weights:", mr.coef_.round(2), "intercept:", round(mr.intercept_, 2))
print("predict (4h study, 7h sleep):", mr.predict([[4, 7]]).round(1))

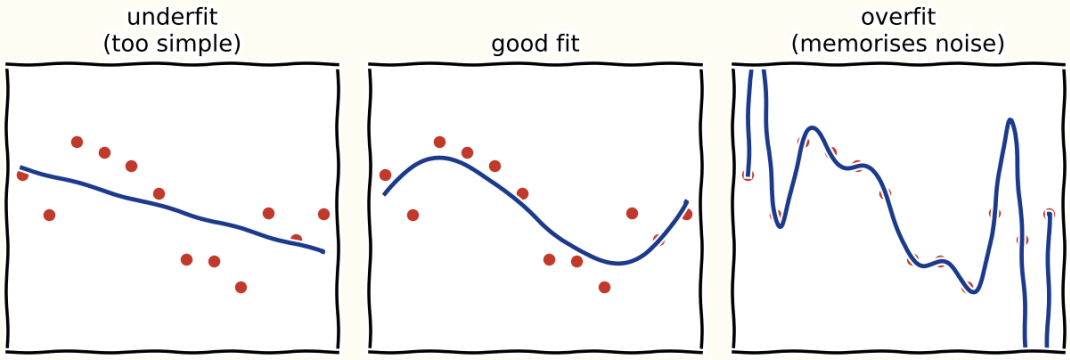
# logistic regression: yes/no answer (pass = 1) - predicts a PROBABILITY, then a class
hours = [[1], [2], [3], [4], [5], [6], [7], [8]]
passed = [0, 0, 0, 0, 1, 1, 1, 1]
lr = LogisticRegression().fit(hours, passed)
print(lr.predict([[2.5], [6.5]]), lr.predict_proba([[4.5]]).round(2))
```

output:

```
weights: [10.11  1.64] intercept: 13.05
predict (4h study, 7h sleep): [64.9]
[0 1] [[0.5 0.5]]
```

Key idea: *Despite its name,* ***logistic regression is a classification model****: it squeezes a straight-line score through the sigmoid function 1 / (1 + e^-z) to get a probability between 0 and 1.*

Overfitting vs underfitting



	Underfitting	Good fit	Overfitting
Model is	too simple	just right	too complex
Training error	high	low	very low (near 0)
Test error	high	low	**high**
Fixes	more features, more complex model	-	more data, simpler model, regularisation, cross-validation

Careful:

- A model that fits the training data perfectly but fails on new data is **overfitting** - it memorised instead of learning.
- Linear regression assumes: a roughly **linear** relationship, independent errors, similar error size everywhere, and no huge outliers.
- **Correlation is not causation:** ice-cream sales and drownings both rise in summer.

Try it yourself:

- Fit a line to your own data (e.g. hours slept vs. mood score) and check R^2.
- Change the noise in the mini-project from 2 to 10. What happens to R^2 and RMSE?
- Fit `LinearRegression` to $y = x^2$ data and observe the poor fit (it is not linear!).

18 Derivative, Chain Rule & Gradient Descent

The maths engine that trains every ML model

What is a derivative?

A **derivative** measures **how fast something changes** - the slope of the curve at a point. dy/dx answers: if x increases by a tiny amount, how much does y change? Speed is the derivative of distance with respect to time.

Formal idea: f'(x) = lim h→0 [f(x + h) - f(x)] / h - the slope of the line through two points that are almost on top of each other.

```
def numeric_derivative(f, x, h=1e-6):
    return (f(x + h) - f(x - h)) / (2 * h)          # slope between two nearby points

f = lambda x: x ** 2
print(numeric_derivative(f, 3))                      # about 6, because d/dx x^2 = 2x
print(numeric_derivative(f, -1), numeric_derivative(f, 0))
```

output:

```
6.000000000838668
-2.000000000002 0.0
```

Rule	Formula	Example
constant	d/dx (k) = 0	d/dx (7) = 0 - a plain number never changes
power	d/dx (x^n) = n · x^(n-1)	x^2 → 2x, x^3 → 3x^2, x → 1
constant multiple	d/dx (k·f) = k · f'	d/dx (5x) = 5, d/dx (4x^2) = 8x
sum	d/dx (f + g) = f' + g'	d/dx (x^2 + 5x + 7) = 2x + 5

Sign of derivative	Function is...	Meaning
positive	increasing (going uphill)	move left to go down
negative	decreasing (going downhill)	move right to go down
zero	flat	possible minimum or maximum - **this is where training wants to be**

The chain rule - a function inside a function

If y depends on u, and u depends on x:

★ **CHAIN RULE**

- dy/dx = dy/du × du/dx
- = (derivative of the OUTSIDE, keeping the inside) × (derivative of the INSIDE)

- **1.** Name the inside: u = ... **2.** Differentiate the outer function in terms of u. **3.** Differentiate u in terms of x.
- **4.** Multiply. **5.** Put the original expression back in place of u.

Function	u (inside)	dy/du	du/dx	dy/dx = dy/du × du/dx
y = (3x + 2)^2	3x + 2	2u	3	2(3x + 2)·3 = 6(3x + 2)
y = (x² + 1)^3	x² + 1	3u²	2x	3(x² + 1)²·2x = 6x(x² + 1)²
y = (2x + 1)^4	2x + 1	4u³	2	4(2x + 1)³·2 = 8(2x + 1)³
y = (5x - 1)^3	5x - 1	3u²	5	15(5x - 1)²

Never trust a derivative you have not checked *- compare it with the numeric derivative:*

```
def num_d(f, x, h=1e-6): return (f(x + h) - f(x - h)) / (2 * h)
checks = [
    (lambda x: (3 * x + 2) ** 2,  lambda x: 6 * (3 * x + 2)),
    (lambda x: (x ** 2 + 1) ** 3,  lambda x: 6 * x * (x ** 2 + 1) ** 2),
    (lambda x: (2 * x + 1) ** 4,  lambda x: 8 * (2 * x + 1) ** 3),
]
for f, df in checks:
    print(round(num_d(f, 1.5), 4), round(df(1.5), 4))
```

output:

```
39.0 39.0
95.0625 95.0625
512.0 512.0
```

Why it works: *The chain rule extends to* ***many layers****: if* $y = f(g(h(x)))$ *then* $dy/dx = f'(g(h)) \cdot g'(h) \cdot h'(x)$ *- one factor per layer. That is exactly what a deep neural network is: many nested functions.*

Note-fix (more precise than the original): *Handy derivatives beyond the notes (used constantly in ML):* $d/dx\ e^x = e^x$, $d/dx \ln x = 1/x$, *sigmoid* $\sigma(z) = 1/(1+e^{-z})$ *has* $\sigma'(z) = \sigma(z)\cdot(1 - \sigma(z))$.

Why we care: it trains ML models

Model: prediction = $w \cdot x + b$. *For one example the* ***loss*** *is* $L = (\text{prediction} - y)^2$. *We want to move* w *and* b *so that the loss becomes smaller - and the* ***derivative says which way is downhill****. Use the chain rule with* $u = w \cdot x + b - y$:

Step	Derivative	Reason
outer	$dL/du = 2u$	$L = u^2$
inner w.r.t. w	$du/dw = x$	$u = w \cdot x + b - y$; *only the* $w \cdot x$ *term has* w
inner w.r.t. b	$du/db = 1$	b *appears once, with coefficient 1*
chain	$dL/dw = 2u \cdot x$	$dL/du \times du/dw$
chain	$dL/db = 2u \cdot 1$	$dL/du \times du/db$

Gradient descent = walking downhill with that slope

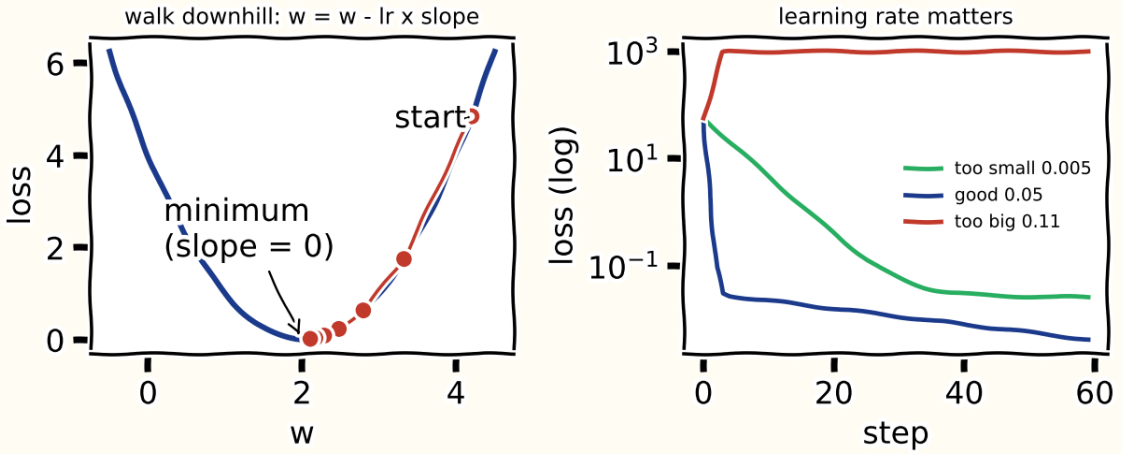
Update rule: $w = w - lr \times dL/dw$, $b = b - lr \times dL/db$. *The* ***learning rate*** lr *is the step size: too big and you overshoot and explode, too small and training crawls. For the whole dataset we* ***average*** *the gradients over the* n *examples.*

```
w, b, lr = 0.0, 0.0, 0.05
xs = [1, 2, 3, 4, 5]
ys = [3, 5, 7, 9, 11]                          # the hidden rule is y = 2x + 1
n = len(xs)
for step in range(2001):
    dw = db = loss = 0
    for x, y in zip(xs, ys):
        u = w * x + b - y                      # error
        loss += u * u
        dw += 2 * u * x                        # chain rule for w
        db += 2 * u                            # chain rule for b
    if step in (0, 1, 2, 3, 10, 100, 500, 2000):
```

```
    print(f"step {step:4d}  w={w:6.3f}  b={b:6.3f}  loss={loss / n:9.5f}")
  w = w - lr * dw / n                                  # small step against the slope
  b = b - lr * db / n
print(round(w, 2), round(b, 2))
```

output:

```
step   0  w= 0.000  b= 0.000  loss= 57.00000
step   1  w= 2.500  b= 0.700  loss=  1.94000
step   2  w= 2.040  b= 0.580  loss=  0.09320
step   3  w= 2.122  b= 0.610  loss=  0.03034
step  10  w= 2.097  b= 0.651  loss=  0.02221
step 100  w= 2.021  b= 0.925  loss=  0.00103
step 500  w= 2.000  b= 1.000  loss=  0.00000
step 2000  w= 2.000  b= 1.000  loss=  0.00000
2.0 1.0
```



Left: the slope is large far from the minimum (big steps) and shrinks to 0 at the bottom (tiny steps) - gradient descent slows down by itself. Right: with this data, lr = 0.05 converges quickly, 0.005 is slow, and 0.11 makes the loss explode (the curve is cut off at 1000 for the chart).

Backpropagation = the chain rule, repeated

A neuron with a sigmoid: $z = w \cdot x + b$, $out = \sigma(z)$, $L = (out - y)^2$. Three nested functions → three factors: $dL/dw = dL/dout \times dout/dz \times dz/dw$.

```
import math
x, y, w, b = 2.0, 1.0, 0.5, 0.1
sigmoid = lambda z: 1 / (1 + math.exp(-z))

z = w * x + b                                          # forward pass
out = sigmoid(z)
L = (out - y) ** 2

dL_dout = 2 * (out - y)                                # backward pass: multiply the local derivatives
dout_dz = out * (1 - out)
dz_dw = x
dL_dw = dL_dout * dout_dz * dz_dw

h = 1e-6                                               # numerical check of the same derivative
Lp = (sigmoid((w + h) * x + b) - y) ** 2
```

```
Lm = (sigmoid((w - h) * x + b) - y) ** 2
print(round(dL_dw, 6), round((Lp - Lm) / (2 * h), 6))
```

output:

```
-0.187175 -0.187175
```

Key idea: ***Big idea:*** *deep learning is gradient descent where the gradients come from the chain rule applied layer by layer from the output back to the input (backpropagation). Everything in this PDF - lists of numbers, NumPy arrays, functions, loops, derivatives - is a building block of that one loop:* ***predict → measure loss → compute gradients → update weights → repeat.***

Try it yourself:

- *Differentiate* `y = (4x - 3)^5` *and verify numerically with* `num_d`.
- *Run the gradient-descent code with* `lr = 0.11` *and watch the loss blow up; then try* `lr = 0.001` *with 2000 steps.*
- *Add a third weight: fit* `y = 2x + 1` *using only* `w` *(no* `b`*) and see why the loss cannot reach 0.*

19 Quick Revision Sheet

One page to read the night before the exam

Topic	Remember	Topic	Remember
int float str bool	number / decimal / text / True-False	list [ ]	ordered, changeable, duplicates ok
// and %	floor divide / remainder	tuple ( )	ordered, fixed
**	power: 2 ** 3 = 8	dict { k:v }	find by key, fast lookup
and or not	both / any one / flip	set { }	unique items only; empty set is set()
for vs while	known repeats / until condition	open modes	r read, w write (erases!), a append
break continue	leave loop / skip to next round	with open(...)	closes the file automatically
def + return	make function, send result back	try/except/finally	attempt / handle / always run
*args **kwargs	tuple of extras / dict of extras	class / object	blueprint / real thing
lambda	tiny one-line function	__init__ / self	constructor / the object itself
scope	Local → Enclosing → Global → Built-in	4 pillars	encapsulation, inheritance, polymorphism, abstraction
list comprehension	[expr for x in it if cond]	regression	predicts a number; $y = m \cdot x + c$
NumPy	vectorised maths on arrays	MSE	average of squared errors; R^2: 1 perfect, 0 = guess the mean
a is b vs a == b	same object / same value	chain rule	$dy/dx = dy/du \cdot du/dx$
slice	s[start:stop:step], stop excluded	gradient descent	$w = w - lr \cdot slope$

Formula box

Idea	Formula
Line	$y = m \cdot x + c$
Least squares	$m = \Sigma(x - \bar{x})(y - \bar{y}) / \Sigma(x - \bar{x})^2$, $c = \bar{y} - m \cdot \bar{x}$
Error metrics	$MSE = mean((y - \hat{y})^2)$, $RMSE = \sqrt{MSE}$, $MAE = mean(\|y - \hat{y}\|)$, $R^2 = 1 - SS_{res} / SS_{tot}$
Power rule	d/dx x^n = n·x^(n-1)
Chain rule	$dy/dx = dy/du \cdot du/dx$
Gradients for $L = (w \cdot x + b - y)^2$	$dL/dw = 2u \cdot x$, $dL/db = 2u$ with $u = w \cdot x + b - y$
Gradient descent	$w \leftarrow w - lr \cdot dL/dw$, $b \leftarrow b - lr \cdot dL/db$
Backprop	$dL/dw = dL/dout \cdot dout/dz \cdot dz/dw$

Common mistakes (and the fix)

Mistake	What happens	Fix
= instead of == in a condition	SyntaxError / wrong logic	== compares, = assigns
Missing : or wrong indentation	SyntaxError / IndentationError	colon after if/for/while/def/class, 4 spaces
Not converting input()	TypeError or "5" + "5" = "55"	int(input(...))
Opening a file with "w" to add data	old data destroyed	use "a"
Forgetting self	TypeError: takes 0 positional arguments	def method(self, ...), self.attr
while that never changes its condition	infinite loop	update the variable; Ctrl + C to stop
b = a for a list	both names change together	b = a.copy()
x = x.sort()	x becomes None	x.sort() or x = sorted(x)
range(1, 5) for 1..5	off by one	range(1, 6)
Mutable default def f(x, L=[])	list shared between calls	L=None then create inside
Comparing floats with ==	0.1 + 0.2 != 0.3	math.isclose or round
{} for an empty set	creates a dict	set()
Training and testing on the same data	falsely great score (overfitting)	train_test_split
Learning rate too large	loss grows / nan	lower lr, scale features

20 Practice Problems

Try first, then peek at the hints

#	Area	Problem	Hint / answer
1	Variables / types	Swap two numbers without a temp variable. What is type(3 / 1)?	a, b = b, a; float.
2	Strings	Check if a string is a palindrome ignoring case and spaces.	t = s.replace(' ', '').lower(); t == t[::-1]
3	Operators	Convert 3725 seconds to h:m:s.	h = 3725 // 3600, m = 3725 % 3600 // 60, s = 3725 % 60 → 1:2:5
4	Conditionals	Read three numbers, print the largest without using max.	chain if a >= b and a >= c ... elif ... else
5	Loops	Print all prime numbers below 50.	nested loop with for ... else, or is_prime() from Topic 8
6	Loops	Sum the digits of 9475 using while.	total += n % 10; n //= 10 → 25
7	Functions	Write average(*nums) returning 0 for no arguments.	return sum(nums) / len(nums) if nums else 0
8	Recursion	Write power(base, exp) recursively.	base case exp == 0 → 1, else base * power(base, exp - 1)
9	List	Remove duplicates but keep order.	loop with a seen set, or list(dict.fromkeys(items))
10	List	Second largest element without sorting.	track largest and second in one loop
11	NumPy	Standardise [2,4,6,8] to mean 0 and std 1.	(a - a.mean()) / a.std()
12	Tuple / dict	Return (min, max) and store results per student in a dict.	stats = {name: min_max(marks)}
13	Dict	Count word frequency and print the top 3.	count[w] = count.get(w, 0) + 1, then sorted(..., key=...)
14	Set	Common and unique hobbies of two friends.	a & b, a ^ b
15	Files	Append timestamps to log.txt each run and show how many lines it has.	datetime.now(), mode "a", count lines
16	Classes	Build Rectangle with area(), __str__, and __eq__.	compare areas inside __eq__
17	Pillars	Abstract Employee with salary(); implement Manager and Intern.	ABC + @abstractmethod; override in the children
18	Regression	Fit a line to (1,2),(2,4.1),(3,5.9),(4,8.2) by hand with the formula, then verify with sklearn.	m = 2.04, c = -0.05 approximately ($\bar{x}$=2.5, $\bar{y}$=5.05)
19	Derivatives	Differentiate y = (4x - 3)^5.	20(4x - 3)^4
20	Gradient descent	Modify the loop to also stop when abs(dw) < 1e-6.	if abs(dw / n) < 1e-6: break

Mini projects to combine everything

- **Student manager** - dict of students, a Student class, save/load marks in a file, show the topper and class average.
- **Number-guessing game** - loops, conditionals, random, functions, play-again menu.
- **Marks predictor** - collect (hours, marks) pairs, fit LinearRegression, report R^2, predict for new hours.
- **Gradient descent from scratch** - fit a line to your own data using only loops, then compare with scikit-learn.

A 4-week study plan

Week	Focus	Topics	Goal
1	Core language	1–8	solve 15 small loop / function problems without looking
2	Data structures + files	9–14	build the student manager (lists, dicts, files)
3	OOP + NumPy	10, 15–16	rewrite the manager with classes; do 10 NumPy exercises
4	ML maths	17–18	code regression and gradient descent from scratch, then with scikit-learn

Key idea: You now have the full chain: **Python basics → data structures → files → OOP → NumPy → regression → derivatives → gradient descent.** Next steps: pandas for tables, matplotlib for plots, then classification and neural networks (PyTorch / TensorFlow).